॥वसुधैव कुटुम्बकम्॥

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

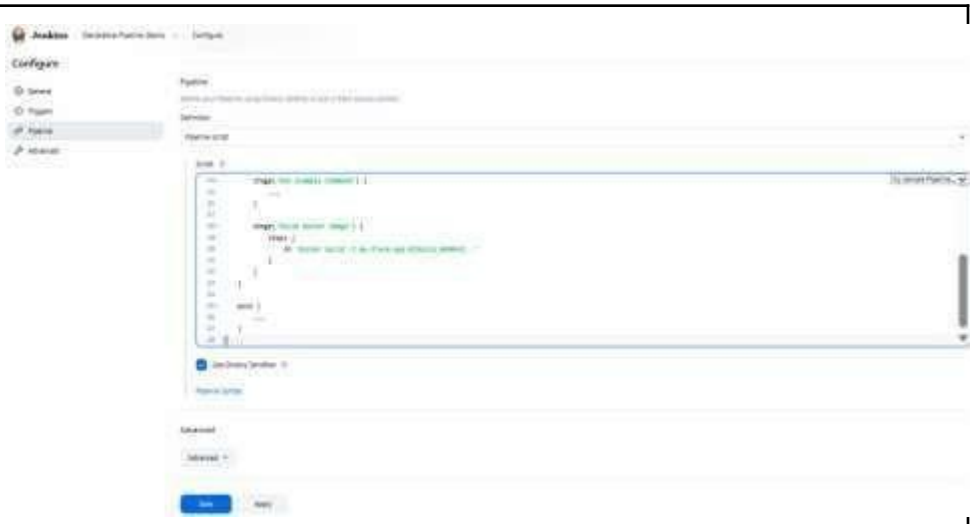
Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category – I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 14

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	**Raghav Sharma**
PRN No.	**23070122171**
Branch	CSE, Batch (2023-27), L3
Academic Year & Semester	25 – 26, Sem VII
Date of Performance	23rd July, 2026
Title of Assignment:	CI/CD: Jenkins Pipeline building Docker Image
Theory:	A Jenkins Pipeline automates the Continuous Integration and Continuous Deployment (CI/CD) process by executing a series of predefined stages such as source code checkout, build, test, and deployment. In this experiment, a Docker image was built automatically using Jenkins after fetching the application source code from a GitHub repository. The docker build command created a versioned Docker image using the Jenkins build number as the image tag, ensuring unique image versions for every pipeline execution. This integration of Jenkins and Docker enables consistent, repeatable, and automated application builds, reducing manual effort and improving software delivery efficiency. **Commands Used** • `docker build -t my-jenkins-docker .` • `docker run -d --name myjenkins --restart unless-stopped -u root -p 8080:8080 -p 50000:50000 -v jenkins_home:/var/jenkins_home`

	`-v /var/run/docker.sock:/var/run/docker.sock my-jenkins-docker` • `docker exec -it myjenkins docker --version` • `docker exec -it myjenkins rm -rf /var/jenkins_home/workspace/Declarative-Pipeline-Demo` • `git add .` • `git commit -m "Add Dockerfile for Jenkins pipeline"` • `git push origin main` • `docker images`
Implementation:	**Step 1: Access and Edit Your Pipeline Configuration**  Accessed Jenkins Pipeline Configuration — Opened the existing Declarative Jenkins Pipeline and navigated to the Pipeline configuration section to modify the pipeline script. **Step 2: Add the Build Docker Image Stage**



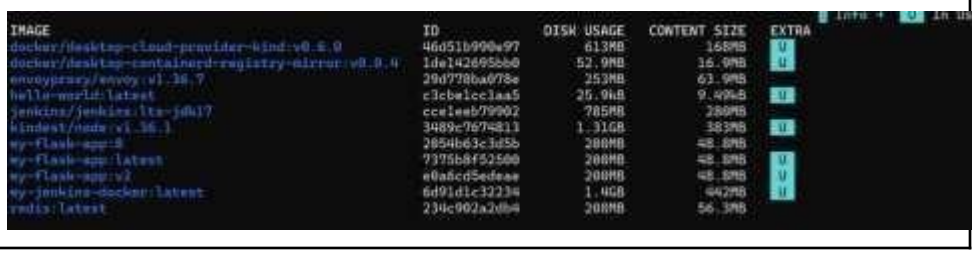
Added Docker Build Stage — Modified the existing Declarative Jenkins Pipeline by adding a new stage to build the Docker image using the `docker build` command with the Jenkins build number as the image tag.

Step 3: Run the Pipeline

```
#10 exporting to image
#10 exporting layers
#10 exporting layers 1.2s done
#10 exporting manifest sha256:62529d2d144f57b03dcf86255a5a08c7df3cb943bf5ed40cea2570d545d7d790 0.0s done
#10 exporting config sha256:00c5c217de619d1e4df43415c660e77578e66ee50015b4700ca808f586fcf2e5 0.0s done
#10 exporting attestation manifest sha256:4de317e81b7b780c8ed2a2e1a0b466b5a14a84b3a8e07029963b6b78e9dcc663
#10 exporting attestation manifest sha256:4de317e81b7b780c8ed2a2e1a0b466b5a14a84b3a8e07029963b6b78e9dcc663 0.1s done
#10 exporting manifest list sha256:2054b63c3d5bd284d8ca521bfd8fb447d8cf99798cac3cce2644b04d7e29bbb3 0.0s done
#10 naming to docker.io/library/my-flask-app:8 done
#10 unpacking to docker.io/library/my-flask-app:8
#10 unpacking to docker.io/library/my-flask-app:8 1.0s done
#10 DONE 2.5s
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Pipeline finished.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Executed Jenkins Pipeline — Successfully executed the Jenkins Declarative Pipeline containing the Docker image build stage. The pipeline completed successfully, and the Docker image was built and tagged without errors.

Step 4: Verify the Docker Image



	Verified Docker Image — Verified that the Docker image (`my-flask-app:8`) was successfully built by the Jenkins Pipeline and stored in the local Docker image repository using the `docker images` command.

Subject Faculty Name: Prof. Arijit Dutta